

Grading forecasts against the exact truth

Design document for a hard-but-fair Harbor task. t20-exact-forecast. Siddharth Shashank Kumar.
September 2026.

1. Summary

The brief asks for one new, long-horizon task in Harbor format that a frontier coding agent fails for a fair and attributable reason. This document records what I built, why each part is built the way it is, what the experiments showed, the two architectures I built first and set aside, and what happened when the two named models were run on the task under a rule I had committed before either of them ran.

The task is a forecasting problem with the luck taken out of the grade. An AI agent is given the ball-by-ball history of a Twenty20 cricket league that was played inside a computer. The league has invented players, and each player has a hidden true ability. The agent also gets the program that plays the matches, with those abilities left out, and the fixtures and line-ups for the coming season. It has to write a program that gives the home side's chance of winning each fixture. I built the league, so I know every hidden ability, and that lets me work out the true chance for every match. I grade the agent's chances against those true chances. No match is played for grading. A forecast cannot get lucky or unlucky; it is only close to the truth or far from it.

The simulator is calibrated to 295,557 real balls from the Cricsheet IPL archive, and only aggregate constants ship with the task. Every file in this repository was typed by me and checked against a known number before the next one was written. The checks caught eight bugs that produced no error and returned plausible output, and they caught two mistakes in my own documents. The notes that accompany the repository record what each file does, why, and what its check caught.

- **Measured.** Summed over the eight graded worlds, every careless approach has at least 1.50 times the regret of a reference built from ordinary regularised statistics, and a coin flip has 1.89 times. The reference passes the task's own gate at exactly 1.000 times its stored regret.
- **Measured, and it changed the design.** A bar analysis on eight worlds that are never graded showed that my first pass rule, a bar on every world, was unsound. Simulation noise reached 5.9 percent of the reference's regret on one world, and on one world in eight a coin flip beat the reference. The rule became a bar on the total over eight worlds (Section 5.16).
- **Measured on the task.** With the rule committed before any model ran, Claude Opus 4.7 under Claude Code and GPT-5.5 under Codex, both at high reasoning effort, each ran five times. All ten failed, at 1.11 to 1.70 times the reference's regret, nine of them with the fingerprint of a forecaster that believes small samples. Every run wrote a valid, deterministic forecast inside its constraints, so every verdict is about forecast quality and nothing else. Reading the programs found the cause in every one, prior scales set 3 to 44 times too weak without a check that could see it, and changing that one constant moved each program most of the way to the reference (Section 6).

The central principle of the whole design is this: I grade probability estimates against the hidden probability itself, not against one random realisation of that probability. This removes outcome luck from the verdict and makes unjustified confidence expensive. Everything else in the document is in service of that principle, or of making the task fair enough that a failure against it can be attributed to the agent.

2. The brief and how I read it

The brief says the primary deliverable is a clear design document, that a production-grade or end-to-end system is not required, and that diagrams, pseudocode, small prototypes and focused experiments should be used only where they help explain or validate a decision. I have followed that literally. Every prototype below exists because a decision depended on a number I could not get any other way. The rubric lines and where this document answers each:

Rubric line	Points	Where this document answers it
Harbor compliance and reproducibility	20	Sections 5.17 to 5.22 and 7. Pinned images by digest, locked libraries, shipped data, both gates passed, the oracle at exactly 1.000.
Fairness and solvability	20	Sections 5.18, 5.19 and 5.23. Public structure, hidden values, a reference that uses only the agent's files, a rule committed first.
Verifier quality	20	Sections 5.7, 5.20 and 5.21. Exact grading against known truth, held-out worlds, a separate container, an unprivileged runner, no judge.
Long-horizon difficulty	15	Section 5.24. Estimation, validation, simulation and generalisation are coupled, and an early wrong choice surfaces only in the final number.
Evidence of frontier-model failure	15	Sections 6.1 to 6.3. Both named models, native harnesses, five trials each, 0 of 5 and 0 of 5, the cause identified by fingerprint and confirmed by one-constant ablation.
Originality and realism	10	Sections 5.1 to 5.12. Exact-truth grading of forecasts, with a simulator calibrated to a real archive and constants reproducible from raw data.

I take hard but fair to mean four things that can each be checked.

- A program decides pass or fail. There is no human or model judge, and no match result or other noisy outcome enters the grade.
- Everything the grader checks is stated in the handbook, and everything needed to succeed is in the files the agent receives.
- A competent practitioner can solve it with standard methods. An oracle that uses only the agent's files demonstrates this by passing the gate.
- A failure names its cause. The verifier reports which known tier a submitted forecast most resembles, and the reward keys separate a bad forecast from a broken program.

The rule I hold the task to throughout is short. Unknown quantities are allowed. Unknown rules are not. The hidden player quality is not disclosed because estimating it is the task. The existence of player quality is disclosed. The hidden form timescale is not disclosed because estimating temporal persistence is part of the task. The existence of drifting form is disclosed. The exact future probabilities are hidden because they are the answers. The scoring rule used to compare against them is fully disclosed. A failure should therefore come from inference, modelling, implementation or verification, not from an undisclosed rule.

3. What the experiments taught me

Six findings changed the design. Each came from a run, and each contradicted what I expected before the run.

- **Difficulty that lives in a written rule does not beat the top tier.** My first architecture stated one rule and planted forty-eight violations of it across a repository. Opus 4.7 and GPT-5.5 repaired every one, every time. A capable agent implements anything it can read, however many files it is spread across.

- **A season of cricket is mostly luck.** A split-half test on the real archive gave a full-season reliability of 0.46 for a batter's strike rate and 0.27 for a bowler's economy. Batters appear to differ by 0.20 runs per ball and really differ by about 0.14. Whoever believes small samples will be confidently wrong, and the logarithmic score charges most for exactly that. This number is the reason the task exists.
- **An effect is real only if it repeats in independent data.** Grounds repeat at 0.71. A batter against a specific bowler repeats at 0.18, on the 113 best-sampled pairs. Head-to-head records, the thing a fan reaches for first, are mostly noise. The simulator contains what repeats and refuses to model what does not.
- **Benchmark difficulty is a random variable.** From three worlds I concluded the reference's skill was stable at about 0.6. Over eleven worlds it ranged from -0.21 to 0.73. Some worlds have little to predict, and there a plug-in forecaster's estimation noise costs more than its signal earns. Three worlds were too few.
- **A threshold cannot repair a statistic whose variance is too high relative to the separation you care about.** No per-world tolerance could both absorb the reference's own simulation noise and reject the nearest careless tier. The fix was not a cleverer percentage but a different aggregation unit.
- **A test is only as good as the number it is compared against.** One expected value in my own checks was guessed before it was computed, and the guess was wrong while the code was right. A stale table in this document was found by rerunning a twenty-second validation script rather than reading the numbers off the page.

From these I wrote down what a winning task needs. The answer must be known by construction. The specification must fit in one sentence so that ambiguity cannot be the difficulty. The agent's own tests must be weak evidence of success while the grader's check stays exact. The steps must be coupled so that an early mistake shows up in the final number. And every number in the world must have a stated provenance, measured or chosen, so that a reviewer can disagree with a choice without that disagreement touching a measurement.

4. Architectures considered

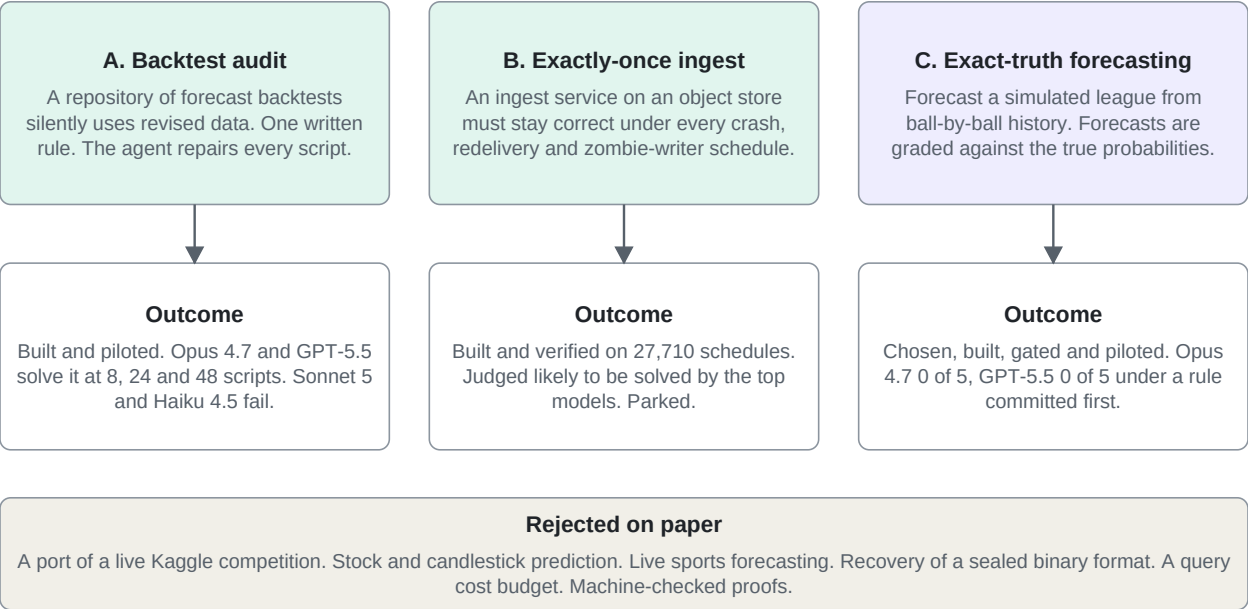


Figure 1. The three architectures I built or prototyped, what happened to each, and the ideas I rejected before building anything.

	A. Backtest audit	B. Exactly-once ingest	C. Exact-truth forecasting
The agent must	Find and repair every use of information that did not exist on the forecast date	Repair a service so readers always see whole batches, under every declared fault	Learn hidden skills from history and forecast fixtures
Difficulty comes from	A written rule silently broken in many places	Rare interleavings and crash points	Statistical inference against a planted truth
Verifier	Honest twin of every script on three worlds, row by row	Every schedule inside declared bounds, guarantee checked before every step	Expected log loss against exact win probabilities on held-out worlds
Status	Built, gated, piloted	Built, verified locally, never piloted	Built, gated, piloted
Verdict	Separates tiers. Top tier passes.	Excellent verifier. Judged too easy for the top tier.	Chosen.

4.1 Architecture A. A version-faithful backtest audit

Surveillance counts are revised after they are first published. A forecast backtest that reads today's settled numbers, and not the numbers that existed on each forecast date, manufactures skill that was never there. This is my day job. The task gave the agent a repository of N backtest scripts, shared helpers with silent pandas defaults, and decoy analyses that use settled data on purpose, under one rule: a forecast issued on a date may depend only on revision-log rows whose issue date is on or before that date. The verifier reran every script beside an honest twin on three worlds, one of them the visible world with its future removed, and compared forecasts row by row. A script that still read settled data changed its forecasts when the future was removed, so it could not hide.

Run (real Harbor harness)	Scripts	Result
Claude Opus 4.7, Claude Code, high effort	8, 24, 48	1.000 every time
GPT-5.5, Codex, high effort	24, 48	1.000 every time
Claude Sonnet 5	24	0 of 24. Every planted violation fixed; one shared default missed; every backtest 95 to 98 percent right; the wrong numbers reported faithfully.
Claude Haiku 4.5	24	0 of 24. Whole families of violations left in place.
Oracle and do-nothing gates	8, 24, 48	1.000 and 0.000

Sonnet's failure is the instructive one, a fair failure of verification. But it is not a failure of either named model. Architecture A is a capability ladder, and it taught me that a task beats a capable solver only through an asymmetry: an information advantage that is cheap for the builder to create, exact to check and expensive to invert. A planted truth is one.

4.2 Architecture B. Exactly-once ingest under a declared fault model

A small service ingests uploads from wearable devices into day tables on an object store that offers whole-object writes and put-if-absent, and nothing else. The guarantee is one sentence: at every moment a reader must see the batch result of some set of whole batches that includes every acknowledged batch. A deterministic scheduler owns every interleaving, so an explorer can enumerate them; the reference service passes all 27,710 schedules in under two minutes, and each of five planted defects is caught with its own fingerprint. I parked it because the agent also receives a random-schedule runner, the service is sixty lines, and iterate-until-green is what these models do best. I judged a pass more likely than a fail and stopped before spending pilot time on it.

4.3 Ideas rejected before building

Idea	Why I rejected it
Port of a live Kaggle competition	The brief excludes ports. A leaderboard metric has no correct answer, so any pass mark is arbitrary.
Stock or candlestick prediction	Scored against realised prices, the grade is mostly luck, and historical prices are in every model's memory.
Live sports forecasting	Results arrive after the deadline, one tournament is a tiny sample, and the ranking is decided by luck.
Recovery of a sealed binary format	Close to existing reverse-engineering tasks. My real experience here involves a client's files, which I will not use.
Query optimisation under a cost budget	Exact and buildable, but a small synthetic module removes the bottleneck search that makes real performance work hard.
Machine-checked proofs	The cleanest verifier of all. I could not defend its fairness in an interview.

5. The chosen design, part by part

The repository is a pipeline with a public and a private side, and I present it in the order it runs. For each part I say what I am trying to do, how it works, what its check was, what the check caught, and the principle I take from it. Three figures sit in the text where they belong, and seven larger plates at the end of the document draw the whole system, the calibration pipeline, the task build, the Harbor runtime, the grading rule, the research behind the bar, and the pilot results; they are generated from the repository by scripts

under figures/, so they change when the code does.

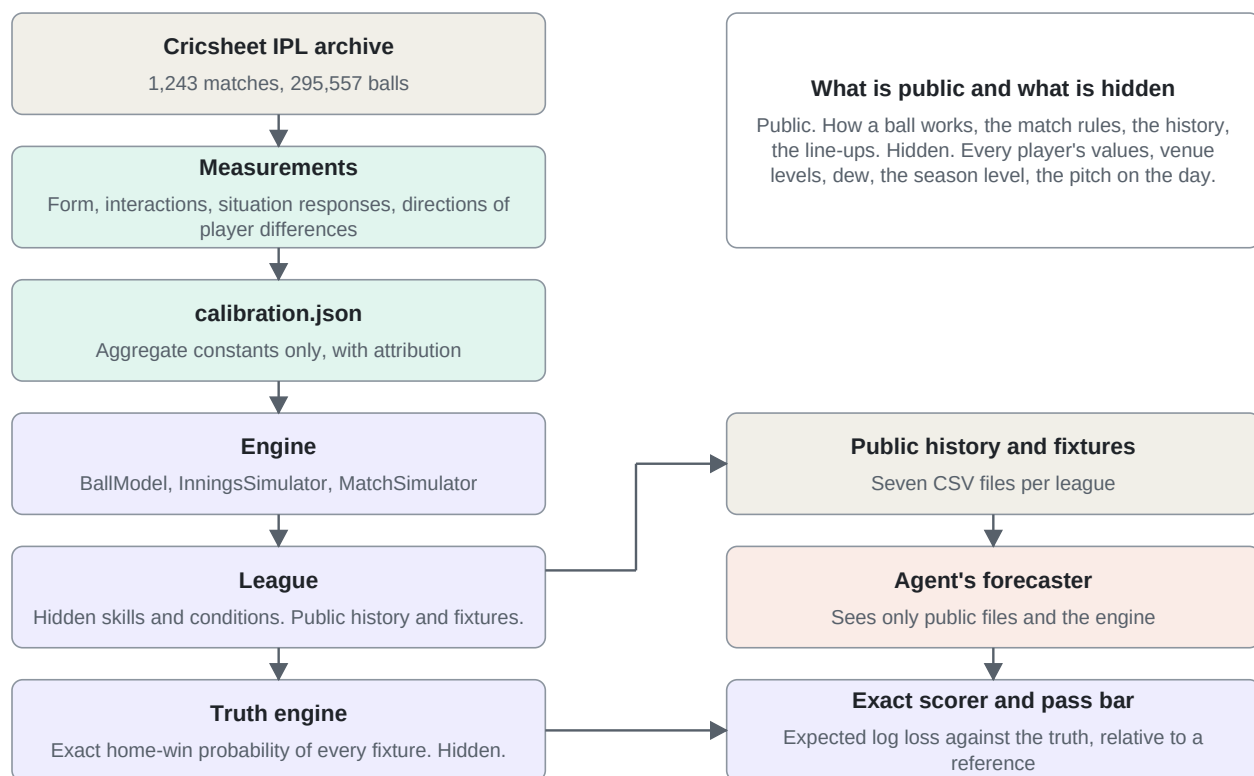


Figure 2. From the real archive to a league I own. The forecaster and the truth engine use the same match engine and differ only in the skills they give it.

5.1 The scorer. Regret against the exact truth

For every future fixture I have two probabilities: the true home-win probability p generated by the hidden world, and the probability q reported by the forecaster. I want the score to measure how much information the forecaster loses by reporting q instead of p . The important advantage of this project is that I own the data-generating process. I do not only observe which team happened to win one realisation of a match; I can simulate the hidden world many times and estimate its underlying win probability very accurately. So I grade forecast probability against true probability, not forecast probability against one noisy match result.

```

# clip the forecast away from 0 and 1 so that one reckless number cannot produce an infinite penalty
q = clip(forecast, 0.002, 0.998)
# expected log loss above the best possible, for one fixture with true probability p
penalty = p * log(p / q) + (1 - p) * log((1 - p) / (1 - q))
# a world's regret is the mean penalty over its fixtures; lower is better, zero only when q = p
regret = mean(penalty)
# skill is the share of the coin flip's regret that the forecast removes; descriptive, never the grading rule
skill = 1 - regret / regret_of_always_saying_one_half

```

The formula starts from the logarithmic score. If the home team wins the loss is minus log q ; if it loses, minus log of one minus q . Normally, with only the observed result, I would compute one of those two values. In my simulator I know the true probability, so I compute the expected log loss directly, which answers: if I could replay this exact fixture infinitely many times under the hidden world, what average log loss would this forecast receive? Even a perfect forecaster pays something, the entropy of the event, because cricket is random. I do not want to punish the forecaster for that. Regret is the expected loss minus the perfect forecaster's expected loss, the part that was the forecaster's own fault, and the algebra reduces it to the

Kullback-Leibler divergence between the two Bernoulli distributions. It is never negative and it is zero only when q equals p .

Why the true probability is uniquely optimal: the expected loss is strictly convex in q with its one minimum at q equal to p , which is what makes the log score strictly proper. A forecaster minimises its expected score by reporting what it actually believes and gains nothing by exaggerating or hedging. Near the truth, regret behaves like squared error scaled by p times one minus p . With p at 0.7, forecasts of 0.5 and 0.9 are both twenty points away, but the cautious one costs 0.082 and the confident one 0.154. That asymmetry is the behaviour I want. Predicting 90 percent when the truth is 70 is a more serious probabilistic claim than calling the match a coin flip, and the scorer should reflect that.

Why I grade against truth instead of results: with 192 graded fixtures, realised log loss carries noise of about 0.014 per match, more than twice the gap I care about between the forecasting tiers. A model could cross or miss the threshold because future teams happened to win or lose. Owning the world removes that problem. Why the log score and not the Brier score: both are strictly proper and both are defensible. I chose the log score because of how it treats confident error, which is exactly the failure I expected in this task, a model overinterpreting noisy player histories or head-to-head records and producing probabilities that are far too extreme. The failed runs showed that behaviour in practice. I do not rely on the locality argument, because for a binary outcome it does not separate the log score from other proper rules. Why I clip: a forecast of exactly zero for an event that can happen makes the regret infinite. The clip at 0.002 bounds a single fixture's penalty at about 6.2 and changes nothing for a sensible forecaster, since the true probabilities live roughly between 0.2 and 0.8. Why skill is descriptive only: a proper score normalised against a baseline is not automatically proper itself, so the pass rule compares regret and skill is a human-readable summary.

The scorer also reports a spread slope, the regression of true logits on forecast logits. A slope of one means the forecasts vary about as much as the truth; below one means overconfidence; above one means the forecasts stay too close to a coin flip. Regret alone cannot describe that distinction as clearly. The check: with truth 0.7, regret of 0.9 is 0.154, of 0.5 is 0.082, of 0.7 is zero. With truths (0.70, 0.40, 0.55) and forecasts (0.90, 0.20, 0.60), mean regret is 0.088, skill is -1.452, mean absolute error 0.15 and slope 0.35. The expected value first written for that second check was a guess, and it was wrong while the implementation was right. I keep that episode because writing an assertion around a guessed number does not create correctness; for mathematical code I want a hand derivation, an independent reference, or a property that must hold, such as regret of the truth being zero.

5.2 From match diaries to a table

Every constant in the simulator has to be measured from real cricket, and every measurement is a counting question. Counting questions are easy on a table with one row per ball and painful on 1,243 match diaries. So the first script converts Cricsheet's YAML match files into one flat table, one row per delivery, one column per property. It computes nothing else. The archive is maintained by Stephen Rushe and released under the Open Data Commons Attribution License; the derived table lives under a directory git ignores, because it can be regenerated and because it contains real player names, which must never ship.

Two decisions in the parser matter later. Runs off the bat and team runs are kept separately, because under the Laws a wide or a no-ball penalty is an extra and not credited to the striker, so batting is measured on one and team totals on the other. And there are two wicket columns: a wicket credited to the bowler, which excludes run outs, and any dismissal that costs the batting side a wicket, which the match situation depends on whoever earned it. Wides and no-balls are flagged because they do not count among the six legal balls of an over; every later rate is per legal ball. Super overs are skipped because they are a

tie-breaker, not ordinary play.

The check was one match against its scorecard, 248 balls, 8 wides, 12 wickets to bowlers, 379 runs, then the whole archive: 1,243 matches, 295,557 balls, 9,871 wides, 1,221 no-balls, 13,466 bowler wickets, 401,423 runs. The first check caught a typo: the dismissal kind was written run_out with an underscore where Cricsheet writes it with a space, so both run outs in the match were credited to the bowlers. Nothing crashed. The scorecard comparison is what caught it, and had it slipped through, every bowler in the calibration would have carried run outs and nobody would have known why the measured spread of bowling skill was wrong.

5.3 Looking before modelling

Before fitting anything I looked at the raw shape of a T20 innings: for each of the twenty overs, the share of balls that are a wicket, a dot, a one, a two, a four or a six, on 125,465 legal balls from 2019 onward. The over number is computed from the count of legal balls before each delivery, not from Cricsheet's label, which counts wides too. The table shows the game's structure directly: over one is half dots and two percent sixes; over ten is nearly half singles; over twenty has sixes at fourteen percent, six times the first over, and wickets at eleven percent, three times the first over. The value of a ball depends on when it is bowled.

The first run came out 717 balls short. The line counting balls before each delivery was missing its subtraction, so every ball counted itself, the last ball of every full innings fell outside the 120 and was dropped, and every over boundary shifted by one. The percentages still looked believable. The total is what exposed it. This is the second silent bug the checks caught, and the reason I compare against known totals after every step.

5.4 Skill against luck

This is the most important measurement in the project. When a batter has a good or bad season, how much of that represents real ability and how much is random variation? For every batter and season I number his matches in order and put the odd ones in one half and the even ones in the other, so that any drift across the season lands equally in both. If scoring rate mostly reflects skill, batters who score quickly in one half score quickly in the other; if it mostly reflects noise, the halves have little to do with each other. The correlation between halves is 0.302. A full season has twice as many balls as a half, so its reliability is higher; the Spearman-Brown formula gives 0.464.

```
r = correlation(odd half, even half) # 0.302
reliability = 2r / (1 + r) # 0.464, Spearman and Brown, 1910
true spread = observed spread * sqrt(reliability) # 0.2047 -> 0.1394 runs per ball
true stability per year = r(season to season) / reliability # 0.36 / 0.46 = 0.78
```

So about 46 percent of the variation in season-level batting rates behaves like persistent signal and the rest like noise. This does not mean 54 percent of every batter's season was luck; reliability describes variance across a population, not a decomposition of one player. Nor does every batter get the same weight: reliability grows with balls faced, roughly as n over n plus k , so a batter seen for 400 balls is trusted more than one seen for 120, and the reference forecaster handles this player by player. The consequence for forecasting is regression toward the mean, which Galton described and which Stein proved is optimal for many estimates at once: an extreme observed number is partly noise, and the estimate should be pulled toward the league average. A forecaster that skips this believes the full observed spread, treats a batter with one strong short period as permanently elite, and is confidently wrong. The reliability applies to scoring rate specifically; the same test gives bowlers 0.27, and wicket-taking barely repeats at all.

5.5 The situation model and the directions in which players differ

This file does two related jobs. First, it learns how the situation of a match changes what is likely to happen on the next ball. Second, after accounting for the situation, it asks how real batters and bowlers systematically differ from an average player. The separation matters: a batter hitting more sixes may be a power hitter, or he may simply have faced more balls at the death when everybody hits more sixes. The model learns the situation first and only then looks for player differences.

The state before every ball is reconstructed from the table: legal balls bowled, wickets lost, runs scored, the target, the batting position. Two derived quantities do the work. Wickets are measured relative to what is usual at that over, so that three down in over four is a crisis and three down in over eighteen is normal. Chase pressure is the logarithm of the required run rate over the rate sides usually manage from that over on, clipped so freak situations cannot dominate; the logarithm makes needing double and needing half equal and opposite. Each ball gets six scores, one per outcome, each a sum of the effects that apply to it, and the softmax turns the scores into probabilities. This is the multinomial logit, the standard regression for a categorical event.

```
z_k = sum_j x_j B_jk          # six scores from 33 situation columns: over, season, innings, wickets, pr
p_k = exp(z_k) / sum_m exp(z_m) # softmax; the row maximum is subtracted first so exp cannot overflow
loss = -sum_i log p_{i, y_i} + (lambda / 2) |theta|^2      # negative log-likelihood plus a tiny ridge
gradient = X^T (P - Y)          # exact, for every softmax model; supplied to L-BFGS
```

Three details each have a reason. Adding the same number to all six scores changes no probability, so one outcome, a single, is fixed at zero as the reference and every other score is measured relative to it. The gradient is supplied exactly rather than approximated, which makes the fit fast and reliable. And the loss is convex in the coefficients, strictly so with the ridge, so it has exactly one minimum and L-BFGS finds it; there are no local minima to worry about. The fitted responses read as cricket: one more wicket lost than usual makes a side defend, dots up 0.128 and sixes down 0.095 on the log-odds scale; one unit of chase pressure makes it attack, sixes up 0.351, dots down 0.392, wickets up 0.191; batting at eight or lower means more dots and dismissals and far fewer boundaries.

With the situation accounted for, I compare each regular player's observed outcomes with what an average player would have produced in exactly the same situations. The log of that ratio, centred, is the player's tilt. Even identical players would have different tilts by chance, so I subtract the covariance that sampling noise alone would produce, the same move as the split-half test applied to six numbers at once, and take the eigenvectors of what remains. The data, not I, decide which player dimensions exist. Among 103 batters with 300 or more balls, 71 percent of the real variation lies on one axis: more sixes, fewer ones and twos, more dismissals, with dots and fours barely moving. That is a power hitter against an accumulator, a style, not a quality. The second axis is mostly about getting out, and that one is quality. Among 110 bowlers the first axis trades fours for sixes, which separates pace from spin. My first engine used a hand-set direction that moved dots and fours; the archive said neither moves on the main axis, so I replaced it. The labels came after the directions.

The bug this file caught is the one I think about most. The gradient's ridge term was typed as plus 0.01 plus theta instead of plus 0.01 times theta. The optimizer followed a slope that disagreed with the loss, stopped after 128 iterations instead of 145, reported success, and returned coefficients with the second-innings and tail-ender rows off by up to 0.04. Nothing crashed. The comparison against known numbers is the only thing that caught it, and a production version would run a finite-difference gradient check as a matter of course.

5.6 The remaining constants, and the rule that an effect must repeat

The main ball model cannot answer the design questions on its own. Do grounds really differ? Are matchups real? How much should a bowler's season be trusted? How many extras occur? What real statistics must the simulator reproduce before I trust it? The rule throughout is that an effect is treated as real only if it repeats in independent data. Something that appears in one half and disappears in the other is noise; something present in both is evidence.

Question	Result in the archive	Design consequence
Extras	0.0764 per legal ball, about nine runs an innings	A league-level constant added on top of the six bat outcomes, not a per-bowler skill
A bowler's season	Reliability 0.27; wicket-taking barely repeats	Bowlers are shrunk harder than batters; the spread of wicket-taking skill is small
Grounds	18 grounds repeat at 0.71; true spread 0.045 runs per ball, about five runs an innings	Every venue gets a hidden scoring level
A specific batter against a specific bowler	Repeat 0.178 on the 113 best-sampled pairs, with each side's level removed within each half	No pair-specific effect. A small pace-versus-spin gap per batter instead.
A batter at a venue	Repeat 0.188 across 309 pairs	A small personal liking for a ground
A bowler at a venue	Repeat -0.026, nothing	No bowler-venue effect
Real targets, 2023 to 2026	First innings 188.5 mean, 37.4 spread, 5.9 wickets to bowlers; chasers win 0.509; chase success 0.81 under 160 falling to 0.21 at 220 and over	Validation targets, never simulator inputs

One detail in the interaction test is easy to get wrong and I got it wrong once. Each participant's own level is removed within the same season and the same half. If the level were computed from all the data, each half's leftover would carry a negative echo of the other half, the two supposedly independent measurements would become dependent, and the repeat correlation would be pushed downward. The conversion from a repeat correlation to a true spread uses r over one minus r , the ratio of real to noise variance under the classical model. The head-to-head result is the one that surprised me least and matters most: it is the same finding as in baseball, where batter-pitcher matchup records contain far less information than fans assume, and it is why the raw head-to-head forecaster is the worst tier on the ladder. This file does not merely compute descriptive statistics. It decides which hidden variables deserve to exist in the simulated world. If an effect repeats, the simulator may represent it. If it does not, the simulator should resist modelling noise as if it were skill.

5.7 The constants file

The merge script does no new measurement. It takes everything measured in the two previous steps, standardises it and converts it into the simulator's units. The season effects from the situation model are compressed into a straight-line trend: its length, 0.0934 per season on the log scale, is how fast the game drifts, and its direction, the way outcomes move as scoring rises, becomes the conditions axis along which venue level, dew, the pitch on the day and the season level all push. That is a modelling simplification I choose deliberately: one well-measured direction over several weakly identified ones. Each player direction is centred, because a common offset carries no softmax information, scaled to unit length, so that one unit means the same distance on every axis, and given a sign by convention, batter style toward more sixes, batter quality toward fewer dismissals, because an eigenvector's sign is arbitrary and a rerun could otherwise flip its meaning.

The directions are then translated into cricket units by differentiating expected runs and wicket probability along each axis at a typical ball. One unit of batter style is worth +0.258 runs per ball and +0.021 wicket

chance: score much faster, accept more dismissal risk, accumulator against power hitter. One unit of batter quality is worth +0.054 runs and -0.047 wicket chance: score somewhat faster while becoming much harder to dismiss. The separation matters because I do not want the simulator to confuse aggression with skill. The spreads along the four player axes come out at 0.347, 0.150, 0.203 and 0.187, and they control both realism and difficulty: too large and the invented players are unrealistically extreme and forecasting is easy; too small and everyone is interchangeable and apparent differences are noise. That is why they are measured rather than guessed.

The output is one JSON file, rounded to four decimals because the underlying measurements do not justify more, with the Cricsheet attribution as its first field and no raw delivery inside it. I think of it as the physics sheet for the artificial cricket universe: the archive is huge, the calibration is compact, and that compression is the point. Above this file I am reconstructing the statistical world from real cricket; below it the simulator can operate without the archive or the fitting code. The check for the whole calibration stage compares this file, block by block, against one produced by an earlier build of the same pipeline. The largest difference anywhere is 0.0007, in the position vectors; most blocks differ by 0.0001 or not at all. The residue is the L-BFGS stopping tolerance carried through, and it changes on different machines.

5.8 Measured against chosen

The loader draws a hard line between numbers I measured from the archive and numbers I chose when designing the world, and represents them with two classes. A measured number should answer: what evidence produced this value? A chosen number should answer: why did I choose it, and what behaviour was I trying to create? Simulation practice separates input modelling from the building of a credible model for exactly this reason; mixing the two kinds in one anonymous list is how simulators come to carry unexamined assumptions. Both objects are immutable, so nothing in the engine can alter a constant by accident and the verifier's check that the engine was left untouched has fewer ways to be fooled.

The chosen numbers are of three kinds, and each has its reason written beside it in the code. Derived from a measurement: a 70 percent talent share with a form memory of 0.75 years gives a year-apart skill correlation of $0.7 \text{ plus } 0.3 \text{ times } e \text{ to the minus one over } 0.75$, which is 0.78, the measured stability; a pace-spin gap of 0.28 makes the public bowling style explain about half the measured type variance; a small batter split against pace and spin because the head-to-head repeat was weak. Set so a validation target is hit: a level constant that recentres the mean total, because players who differ from each other raise the average on their own through the convexity of the softmax; a pitch-on-the-day spread set for the spread of totals; a second-innings wear constant set for the chase rate. Plain judgment: a home lift of about three runs an innings, dew at some grounds, a share of left-handers, and a quarter of players changing teams each off-season, chosen deliberately so that team identity is weak and a model of players keeps information that a model of teams loses. A reviewer can disagree with any of these without that disagreement touching a measurement, and a measured constant can be checked against its source without pretending the design was estimated.

5.9 The engine

The engine turns the ball model into innings and matches, in a way that the true league and every forecaster can share. It knows the public rules of cricket and the public structure of the ball model. It takes the hidden numbers as an argument, the skill book. The true league passes the values it generated; a forecaster passes its estimates; the engine cannot tell which it was handed. That symmetry is how I encode the fairness claim into the architecture. The agent should not have to reverse engineer how a delivery works. I give it the engine. Its job is to infer the hidden quantities that should be passed into that engine.

the six outcomes of a legal ball are w, 0, 1, 2, 4 and 6, and z holds one log-scale score for each

```

z = over_profile[over] + position_shift[batting_position]
z = z + (wickets_lost - usual_wickets[over]) * wickets_vector
z = z + chasing * (second_innings_vector + pressure * pressure_vector)      # the public part, computed once, fixed
z = z + batter_style * d_style + batter_quality * d_quality                # hidden numbers along public directions
z = z + bowler_type * d_type + bowler_quality * d_bowl_quality
z = z + conditions * d_conditions      # venue, season level, pitch on the day, dew, home lift, venue liking
p = softmax(z)

```

An innings is 120 draws from the ball model, played for many copies at once so that each ball is one vectorised step over arrays with one entry per copy. Given the skills, the innings is a Markov chain: each ball depends only on the scoreboard and who is on strike, which is the structure the published Twenty20 simulators use. The outcome is drawn by inversion, one uniform per copy against the cumulative probabilities. Three rules of cricket are in the loop: five bowlers rotate over by over so no one bowls consecutive overs, the batters cross on odd runs and at the end of each over, and an innings ends at ten wickets or when a chase passes its target. Extras are an independent draw at the measured rate. The match simulator plays both batting orders with weight one half each, because the toss is a coin flip and the winner always chases, draws one pitch-on-the-day value per copy that both innings share, and counts a tie as half a win. The win probability is the fraction of copies the home side wins, a Monte Carlo estimate with standard error about 0.006 at 4,000 copies per batting order and 0.0011 at 100,000.

Two design choices are deliberate simplifications. The toss winner always chases and bowlers rotate in a fixed pattern, because either alternative would put a captain's decision policy inside the world that the agent would then have to model too; I remove the problem entirely. There is no fielding, no partnership effect and no ball-to-ball memory beyond the scoreboard, which is the price of an engine simple enough to fit exactly. The skill book is the container for everything hidden, and it has one hook, a pair effect that returns zero, so that a forecaster which believes in head-to-head records has exactly one place to say so without touching the engine. A public-constants object is the boundary of what ships: the profiles, the situation responses, the directions and the extras rate, and not the spreads, because the spreads are the answer to the question the task asks.

The checks: overs one, ten and twenty at the usual wickets reproduce the shape of the over table; a side two wickets down under chase pressure shows the fitted responses; twenty thousand innings of average players give a mean of 189.5 and a spread of 30.0, and 75.6 percent of chases of 170 succeed; two identical sides give a win probability of 0.507, a coin flip within simulation noise, which is the test that catches accidental asymmetry; a home lift ten times the league's real one lifts it to 0.656, which shows the mechanism is connected to the outcome in the expected direction. Two bugs were caught at once: a missing class and a misnamed constant, both by the first import and the first innings. The dangerous bugs in this project are the ones that still return plausible cricket numbers, which is why the statistical checks matter more than these.

5.10 The world

This is where I stop describing the rules of cricket and create an artificial league that I own completely. One random stream seeded once drives every draw: players, skills, handedness, bowling type, venue characteristics, form evolution, transfers, line-ups, fixture order, tosses, pitch conditions and every ball. A seed therefore defines a complete world, not merely the same kind of league, and eight seeds give eight worlds built by the same recipe. Ten squads of the same shape, seven batters, four all-rounders and seven bowlers, make 180 players. Handedness and bowling style are public. Style, bowler type, the two qualities and the pace-spin split are drawn from the measured spreads and hidden. Each quality is fixed talent plus drifting form, split so that the variances add to the measured total.

Form is an Ornstein-Uhlenbeck process. Over a gap of w weeks a share κ equal to $e^{-\kappa w}$ of the old form survives, and fresh noise with variance $1 - \kappa^2$ times the form variance is added, so the total variance never changes and the process is stationary. Form drifts one week at a time during a season, across each off-season before transfers, and once more after the last season. That final off-season is one of the most important pieces of uncertainty in the task: the history ends before it, so every player's current hidden quality contains a component no historical record can reveal, about a quarter of the variance at the forecast date. Even an ideal estimator of the end-of-season state cannot know the fresh innovation. That keeps the true probabilities genuinely probabilistic rather than letting enough history reveal the entire hidden world.

Grounds get a public pitch type and a hidden level, hidden dew at some grounds and a hidden personal liking of each batter for each ground, scaled by the measured venue and affinity spreads and converted from runs per ball into engine units. The history is a double round robin of 90 matches per season in random order over eight weeks, three seasons, 270 matches, about 63,000 balls, each match played with one real copy of the engine, the true skill book of that week, a coin-flip toss, one shared pitch-on-the-day value and a coin flip for a tie. Every ball is logged with the scoreboard before it. The truth engine is the only door to the hidden skills: it plays each of next season's fixtures many times with the true book, with one seed per fixture and chunk so the truth is reproducible and does not disturb the league's own random stream.

The strongest check in the repository lives here. With the earlier build's constants read for comparison, seed 101 produces 270 matches, 62,972 balls, a first-innings mean of 192.11, and a balls table identical, row for row, to the visible world that build had produced. The same players are created, the same matches scheduled, the same tosses drawn, the same ball outcomes. With this repository's own constants the same seed gives 62,972 balls and a mean of 192.12 with a few outcomes flipped, because a fourth-decimal difference in a probability eventually lands a uniform draw inside the tiny interval where two cumulative distributions disagree, and from that ball the innings diverges. This is a useful demonstration of a property of stochastic simulation: fixed seeds give exact reproducibility only when the entire model and random-consumption path are also fixed. It is also why reproducing a calibration to four decimals is not the same as reproducing a particular generated world.

5.11 The airlock

The file reader and writer define the one public representation through which every part of the task sees a world. The task builder, the reference forecasters, the grader and the agent all read the same eight files: every ball with the scoreboard before it, every match, every line-up with batting and bowling slots, the players' public facts, the grounds' public facts, next season's fixtures and their line-ups, and the number of seasons. Nothing hidden crosses this boundary: no skill, no venue level, no true probability. I think of it as the airlock between the private simulator and the public task. Writing controls what passes through; loading guarantees that everyone who enters through the public side reconstructs the same view.

Two details matter. The outcome column is read back as text, because the dot ball is written as the label zero and pandas would otherwise parse it as the number zero while leaving W as text in the same column. And the line-ups carry both a batting slot and a bowling slot as data, so the batting order and the five-bowler rotation are reconstructed from what is stored rather than inferred from row order. The check is byte-level: the seed-101 world, saved to a scratch folder, is identical in all eight files to the folder produced by the earlier build, so the whole chain from raw archive to task folder has been reproduced. The bug caught was a loader whose final return line was missing, so it built everything and handed back nothing; the first call exposed it, which is why end-to-end tests that use the public interface matter more than inspecting intermediate variables.

5.12 Validation against the archive

Individually reasonable components can still produce an unrealistic league once they interact, so I generate three complete leagues, pool their histories, and compare six summaries with the real targets. This is operational validation: not does this function work, but when all the mechanisms run together, does the world produce something recognisably like the system I intended to imitate?

Check	Real IPL 2023 to 2026	Simulated, three leagues
First-innings total, mean	188.5	189.2
First-innings total, spread	37.4	35.0
Wickets to bowlers, first innings	5.9	5.83
Chasing side wins	0.509	0.510
Run rate by over, correlation across the twenty overs		0.977
Chase success, targets under 160 up to 220 and over	0.81 falling to 0.21	0.85 falling to 0.24

Two things are worth recording. First, an ablation refuted a claim I had taken from the literature: adding the response to the target raised the chaser's win rate rather than lowering it, which is why the league carries a small second-innings wear constant. Second, an earlier version of this table carried the numbers from a run made before that constant was raised: 190.6, 35.5 and a chase rate of 0.531. My own validation run found the stale numbers, and this table is the re-validation. The chase rate now sits on top of the archive. The spread of totals remains a little low, and I keep that miss visible rather than adding another hidden noise term to erase it; every new latent mechanism is another hidden quantity that needs justification. Validation is not fully independent of calibration: three of the six quantities were tuning targets, so agreement on them is by construction. The wickets, the run rate by over and the chase success by target were not tuned against, and those are the checks that carry weight.

5.13 The reference ladder

Before I give the task to another model, I build the students who take the exam. A hard task is not useful if every reasonable method gets the same score, nor if even a careful model cannot beat a coin flip. So I build a ladder of forecasters from deliberately naive to fairly careful, each making a recognisable statistical mistake, and score them against the exact truth. The coin flip learns nothing. The team model, a Bradley-Terry fit on results, asks which teams have been good rather than which players will play. The unshrunk player model fits the right structure but trusts small samples. The last-season-only model overreacts to recency and throws away two thirds of the evidence. The raw head-to-head model fits one barely-shrunk number for every batter-bowler pair that ever met. The reference uses the full history with regularisation, and two richer tiers add the small matchup and venue-affinity structures.

The ball-model forecasters fit the engine's own structure backwards. The public part of every ball's six scores is computed once as an offset, and the fit learns only what was added to it: style and quality per batter, type and quality per bowler, level and dew per ground, and a league level, trend, home lift and wear. Because the directions are public the scores are linear in the hidden numbers, so the likelihood is convex and the ridge makes it strictly convex, the same argument as in the situation model. This is penalised maximum likelihood, the posterior mode under Gaussian priors: each ridge strength is one over a prior variance, so a strength of 44 on batter quality is a prior spread of 0.15, which is the true value. How much to shrink is the one thing the reference chooses from the data, and it chooses it the only fair way: fit on the first two seasons, score ball-level likelihood on the third for three candidate multipliers, keep the best, refit on everything. The split is chronological so that nothing from the future leaks into the fit, the held-out season's

level is extrapolated rather than fitted, and the per-match pitch effects are dropped from the held-out score because a new match has none.

Forecaster on the visible world	Regret at the graded settings	Times the reference
Coin flip	0.0208	2.39
Team ratings from results	0.0393	4.52
Players, no shrinkage	0.0120	1.38
Players, shrunk, last season only	0.0096	1.10
Players, shrunk. The reference.	0.0087	1.00
Shrunk, with a raw head-to-head table	0.0478	5.49

The ordering matters more than any fourth decimal. The careful player model clearly beats the naive baselines. Removing shrinkage hurts. Discarding older seasons hurts less. Team-level modelling is poor because team identity is deliberately unstable. Fitting raw pair effects is disastrous because those interactions capture noise. The richer tiers move the reference by about one percent, which means the task is not secretly won by discovering one obscure mechanism; the dominant gains come from understandable decisions, using the public model, estimating players rather than teams, pooling across seasons, and regularising noisy parameters. Summed over the eight graded worlds the careless tiers sit at 1.50, 1.58, 1.89, 3.23 and 4.00 times the reference's total.

One limitation deserves to be explicit here rather than in a footnote. The prior table was chosen with knowledge of the true synthetic population scales. The reference therefore begins with unusually good regularisation scales that an external model is not handed. The chronological validation of the global multiplier is the only part an agent can reproduce. I use the reference as a strong, transparent benchmark whose score I can reproduce exactly, not as evidence that every competent agent should reproduce its methodology. The tolerance in the pass rule softens this advantage; it does not remove it.

5.14 Running the ladder, and the three-world mistake

The ladder runner generates fresh worlds, computes the truth at 10,000 copies, fits every tier, and appends each result to a log. On the first three worlds the reference scored 0.63, 0.57 and 0.61 in skill, the unshrunk tier 1.16 to 1.88 times its regret, last-season-only 1.14 to 2.56, and the head-to-head table 2.7 to 7.6 times and worst of all. That was the ladder shape I wanted, and it was right. From the same three worlds I also concluded that the reference's skill was stable at about 0.6. That was wrong. Across eleven worlds it ranged from -0.21 to 0.73. Some synthetic worlds contain strong, learnable differences; others, by random construction, contain much less signal, and in those a player model that estimates hundreds of latent parameters introduces estimation noise while there is little signal to reward it. It can do worse than saying 0.5 for everything. A negative reference skill is not evidence that the forecaster is broken; it can be a genuine consequence of a weakly predictable realised world.

The error was not in the code but in how much I inferred from three worlds. There are two sources of variation in observed benchmark performance, method quality and world difficulty, and the first experiment measured the first reasonably well while severely underestimating the second. A few favourable seeds do not characterise a stochastic benchmark. I keep this mistake documented because it directly changed the grading design.

5.15 The task data

This is where the simulator becomes an evaluation dataset. For each of eight named worlds, one visible with seed 101 and seven held out with seeds 202 to 808, the build writes the public folder, computes the truth at 100,000 copies per fixture, then reloads the world from the public files, exactly as a solver would, before fitting the reference and the careless tiers. That reload is one of the most important fairness checks in the repository: the generator has access to hidden values, the reference must not, and by discarding the in-memory objects and loading the serialised files I guarantee the reference sees the same balls, matches, players, venues, line-ups and fixtures as an external solution and nothing more. The private folder for each world holds the truth, the reference's regret and the coin flip's, and every tier's forecast for every fixture. That last file exists so the grader can say which known failure mode a submission most resembles.

Why 100,000 copies: each batting order is played that many times, 200,000 matches per fixture, so the worst-case standard error on a probability is the square root of 0.25 over 200,000, which is 0.0011. Regret's sensitivity to the truth is about 0.3 for a good forecaster, so the truth's error moves a fixture's regret by about 0.0003 in a random direction, and averaged over 192 fixtures by about a quarter of one percent of the reference's regret. That is why I call these probabilities exact for grading purposes even though they are Monte Carlo estimates. The reference uses 4,000 copies because it is itself a practical forecasting method and the grader should have far less noise than the thing measured against it. The build is idempotent: a world already built at the requested precision is kept, which is how five worlds were added without rebuilding the first three.

5.16 The pass rule, and the experiment that changed it

I did not want to invent the tolerance after seeing external model results. I wanted to understand the sources of variation first, choose a rule on separate development worlds, and commit it before the pilots. Two quantities matter. Noise: even the same fitted reference, predicted again with a different simulation seed, gives a slightly different regret, and the tolerance must be large enough that this harmless fluctuation cannot fail the reference against itself. Separation: the tolerance must remain small enough that the deliberately careless tiers still fail. The bar analysis measures both on eight worlds, seeds 1001 to 1008, that are never graded, so that even the threshold has its own holdout discipline.

World	Reference regret	Simulation noise	No shrinkage	Last season only	Coin flip
1001	0.0147	2.5%	1.07	1.42	1.19
1002	0.0052	2.6%	1.31	3.44	3.74
1003	0.0066	3.9%	1.16	1.71	2.80
1004	0.0062	5.9%	1.38	2.01	1.42
1005	0.0071	1.8%	2.15	1.95	1.79
1006	0.0107	2.8%	1.36	1.81	2.33
1007	0.0065	4.9%	1.06	2.56	0.94
1008	0.0098	1.2%	1.19	1.68	1.91
Total over the eight	0.0668	1.1%	1.30	1.93	1.90

The last three columns are regret as a multiple of the reference's. My first rule, a tolerance applied on every world, fails twice over. On one world the reference's own re-simulation noise reaches 5.9 percent, so a conservative three-noise tolerance would be about 18 percent; but the nearest careless tier sits only 6 to 7 percent above the reference on two worlds. No number can do both. And on world 1007 the coin flip has lower regret than the reference, so a per-world rule would pass the do-nothing starter on that world. The

problem is not solved by a cleverer percentage; the per-world formulation itself is wrong.

So I changed the statistic. Regret is summed across the eight graded worlds and one tolerance is applied to the total, and the same condition must hold on the seven held-out worlds alone. The worlds are independent, so the relative noise of the sum falls roughly as one over the square root of the number of worlds, about 1.1 percent for eight. In the sum the unshrunk tier is at 1.30 times the reference, last-season-only at 1.93 and the coin flip at 1.90. A tolerance of ten percent is about nine times the noise of the combined score and about a third of the way to the nearest careless tier. It has a noise justification and a behavioural justification. It was written into a small data file that both the grader and the handbook read, and committed before any model ran on this task. If I ran several candidates, saw one score 13 percent above the reference and then decided the tolerance should be 15, the benchmark would no longer be independently evaluating that model; fixing the rule first is the whole point.

5.17 The Harbor contract

Four short files tell Harbor how to run the task. The metadata file carries the slug, which I treat as a stable identity that does not drift once pilots have run against it; the two folders carried from the agent's container into the verifier's, the submission and the engine, so the verifier can grade the first and check the second; a separate verifier environment, so the container the agent modified is never the authority that decides whether its modifications were allowed; two three-hour timeouts, deliberately generous so that the clock can never be the cause of a failure, when the longest pilot used 25 minutes; two CPU cores and four gigabytes, which is what the reference needs and no more; and public networking on the agent side only so the harness can reach its model, since the task itself needs no network. The pass rule lives in one machine-readable file, the tolerance, the absolute slack of zero, the clip and its pre-registration status, so that the grader and the handbook cannot drift apart. The lock file pins numpy, pandas and scipy to exact versions, because this project has already shown that tiny numerical differences make a seeded simulation diverge. The instruction is the agent's first page: the deliverable, its command line, where the history, the handbook and the engine are, the one prohibition, and the grading rule in two sentences.

5.18 The handbook

The handbook is the complete public contract, the fairness argument written in prose. It gives the job and the output format; the scoring formula, the clip, the tolerance and the eight-world rule with its held-out condition; the seven files and their columns, including that the scoreboard columns are before the ball, that the dot is the label zero, that position starts at zero and that target is zero in the first innings; the ball model in words with the engine as the exact definition; a full list of what is hidden and how each hidden thing is built; and the rules the program must follow, determinism, twelve minutes per world on two cores, no network, everything under the solution folder, the engine untouched.

Three sentences do disproportionate work. There is no effect that belongs to one specific batter against one specific bowler: that prevents the head-to-head failure from being an undisclosed trap, so a model that still overfits raw records does so despite being warned. A coin flip has a little under twice the reference's regret: that gives the solver a meaningful scale for its own validation. The engine is yours to use in any way you like, including to test your own method on leagues you simulate yourself: that names the route to a check that can see one's own error. The design measures whether an agent can perform long-horizon work including iterative verification, and I did not want successful self-checking to depend on discovering an undocumented permission. The route should be available; the hard part is using it effectively. What the handbook withholds is every magnitude: the spreads, the form timescale, the venue spread, the size of home lift, dew, wear and the day effect. What exists is public. How large it is, and who has which value, is hidden.

5.19 The starter and the oracle

The starter is the minimum valid answer: it reads the fixtures and writes 0.5 for every one. Its purpose is to make the submission contract impossible to misunderstand, so that the task begins from a working program and the agent's job is to replace the middle of it. Statistically it is the coin flip, and the reward keys are built so that it receives full credit for a valid artifact and satisfied constraints while receiving nothing for forecasting. That split lets me separate did the agent produce a valid submission from did it produce a good forecast, which matters in a long-horizon task where there are many opportunities to break something while iterating.

The oracle is the constructive proof of solvability. Harbor's oracle agent installs the reference forecaster into the solution folder, and that program reconstructs its model from the public league files through the agent's own reader and from the agent's own public constants file, fits, and plays each fixture 4,000 times, the same settings the stored reference numbers were made with. It imports nothing from the generator, the truth engine or the private calibration. A fake oracle that copied precomputed probabilities would trivially score a perfect ratio and prove nothing; this one recomputes the forecasts from the solver-visible side and hands them to the real grader, so its gate result, 1.000 on every key at exactly 1.000 times the stored reference, is an integration test of the whole public path: the engine, the reader, the packaged imports, the dependencies, the seeds and the stored numbers all agree. Its first run scored zero, because the installing shell script had spaces around an assignment, which bash treats as a command, so the oracle was never installed and the verifier graded the starter. A syntax check had passed the file. Only running the gate caught it, which is why the gate is necessary rather than ceremonial.

5.20 The containers

Both images start from the same base, pinned by content digest rather than by tag, so that the same bytes are pulled next year; both fix the Python hash seed; both install the locked libraries. The agent image holds only the public task. The verifier image holds the private grader, the truth and the reference numbers, and it does two things the agent image does not: it creates an unprivileged user, and it removes every other user's right to read the private directory. The submitted program runs as that user. This is least privilege and fail-safe defaults in the sense Saltzer and Schroeder set out: the program gets only the rights its job needs, and access is denied unless granted. An agent benchmark has an adversarial property, because the model is trying to maximise the reward the grader produces, and if there is an easier path to reward through the evaluation machinery a capable optimiser may find it. Reading the private truth would be the clearest example. So there are two layers: the agent never sees the verifier container at all, and submitted code that later runs inside the verifier cannot read its answers. The entry script runs the grader with its output captured and, if no reward file exists afterwards, writes one with zeros, so a grader crash is an explicit zero with its traceback preserved rather than an ambiguous absence.

5.21 The grader

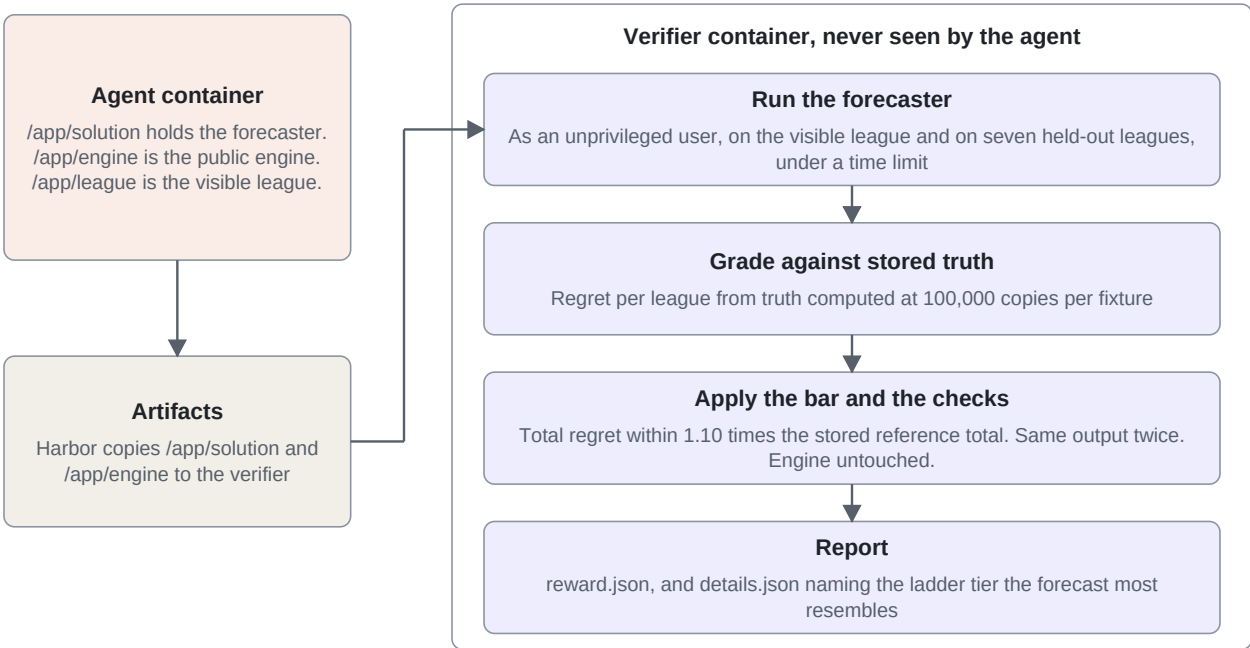


Figure 3. The verifier. The truth and the reference's regret are computed at package time and stored where the agent cannot reach them.

```

# the submission runs in a clean folder beside a pristine copy of the engine, as a user that cannot read /tests
for world in [visible] + seven_heldout_worlds:
    forecast = run('solution/forecast.py', world, user='runner', timeout=720)
    valid[world] = complete and finite and inside [0, 1]
    mine[world] = regret(truth[world], forecast)
    most_like[world] = nearest ladder tier by root-mean-square distance # attribution only, never the verdict
passed = sum(mine) <= sum(reference) * 1.10
passed_heldout = sum(mine[heldout]) <= sum(reference[heldout]) * 1.10
deterministic = run(visible) == run(visible) # to twelve decimals
overall = functional * (0.5 * robustness + 0.25 * constraints + 0.25 * artifact)

```

The grader is the executable definition of success. It runs the submission once per world in a fresh working folder with a stripped environment and a 720-second limit, beside the verifier's own pristine engine; the agent's copy of the engine is only hashed, so modifying it can neither alter the evaluation semantics nor escape the constraint penalty. The switch to the unprivileged user happens when the grader itself runs as root, which is how the verifier image launches it; a grader run as an ordinary user on a laptop has no such boundary, and that is the intended difference between a local check and the real gate. A forecast is valid only if every fixture has a finite number between zero and one; completeness is checked by reindexing against the truth, so a short file cannot pass by omitting fixtures. Regret is the scorer's formula copied in, with the clip read from the rule file. The rule is applied twice, to all eight worlds and to the seven held-out ones alone, and those two verdicts become functional correctness and robustness. The visible world is run a second time and must match to twelve decimals. Constraint satisfaction is the mean of engine untouched, deterministic and inside the time limit; artifact quality is the share of worlds with valid output; overall is functional correctness times the weighted mean of the rest, so nothing counts unless the forecast passes. Nothing in the grader inspects the candidate's code beyond the engine hash, requires a particular method, or compares against my implementation. The accepted artifact is a numeric forecast file. Every path is configurable, so the same grader runs on a laptop, which is how the starter and the oracle were graded before the Harbor gates.

5.22 The packager

Packaging is mechanical, not interpretive. One script assembles the task directory from four source trees, the Harbor files, the engine, the agent-facing sources and the generated data, and places each piece on the correct side of the public-private boundary. Three moves are deliberate. The agent's engine gets a public constants file built from the public-constants transformation, so the spreads and the real targets never enter the agent's container, and the verifier gets a second, pristine copy of the same engine to hash against. The handbook's three placeholders are filled from the rule file, so the tolerance the agent reads is the tolerance the grader applies by construction rather than by care. And the oracle's forecaster is written from the ladder file with one import line rewritten, so the solvability witness cannot silently drift from the model that produced the stored reference numbers. The visible world goes to the agent side, all eight to the verifier side, the private files to the verifier only. A fresh clone rebuilds the directory byte for byte, which is what the reproducibility claim means.

5.23 Why it is fair

- Nothing about how a ball works is left to interpretation, because the engine ships as code and the public constants file is generated from the same object the generator uses.
- The agent is told the scoring rule, the clip, the tolerance, the eight-world rule, the time limit and the determinism rule, and the rule it reads is the rule the grader applies by construction.
- The two main traps, believing small samples and believing head-to-head records, are documented facts in the handbook, and the yardstick for self-checking is in print.
- The reference uses only the agent's files and the agent's engine, is loaded through the same reader the agent gets, and is ordinary statistics. It passes the gate at exactly 1.000.
- The agent may use the engine to simulate its own leagues and test its own method against a truth it constructs. The handbook says so in one sentence.
- The verdict is numeric and the artifact is a two-column file. No judge, no string matching, no inspection of the agent's method.

5.24 Why it is hard, and how a failure would be read

- Two numbers per batter and two per bowler must be learned from a few hundred balls each, so shrinkage is essential. A fit at a default penalty lands near the unshrunk tier and fails.
- Strike rate rewards style. Winning depends on style, quality and the match situation together, so forecasts have to come from simulation and not from a table of averages.
- A backtest on ninety matches cannot separate the tiers: realised log loss on that many results has a standard error near 0.015 per match against a coin-flip-to-reference gap of 0.009. The agent's own validation is weak evidence, and the grader's is exact.
- The steps are coupled. A leak in validation picks the wrong shrinkage, which makes the fit overconfident, which the log score charges for.
- The last off-season is unobserved, so even a perfect estimator of the end-of-season state faces irreducible uncertainty, and the truth stays probabilistic.
- The reference holds an advantage no agent has, its starting prior spreads. The ten percent tolerance softens this; it does not remove it.

6. Pilots on this task

The pass rule was committed to the repository before any model ran. All ten runs used the real Harbor harness, the task's own containers and a three-hour agent budget, alternating between the two models.

Run	Model	Total regret, times the reference (bar 1.10)	Held-out worlds only	Forecast most resembles	Session
1	Claude Opus 4.7, Claude Code, high effort	1.533	1.492	no shrinkage on 6 of 8 worlds	16 min
2	GPT-5.5, Codex, high effort	1.575	1.613	no shrinkage on 7 of 8	25 min
3	Claude Opus 4.7	1.573	1.537	no shrinkage on all 8	25 min
4	GPT-5.5	1.545	1.543	no shrinkage on 5, last season only on 2	19 min
5	Claude Opus 4.7	1.396	1.376	no shrinkage on all 8	21 min
6	GPT-5.5	1.109	1.117	the reference on 7 of 8	20 min
7	Claude Opus 4.7	1.364	1.345	no shrinkage on all 8	22 min
8	GPT-5.5	1.362	1.341	no shrinkage on all 8	13 min, cut short (see below)
9	Claude Opus 4.7	1.703	1.661	no shrinkage on all 8	23 min
10	GPT-5.5	1.409	1.398	no shrinkage on all 8	26 min

Claude Opus 4.7 passed 0 of 5 and GPT-5.5 0 of 5. No run came near a time limit; every run wrote a complete, valid, deterministic forecast and left the engine untouched, so every verdict is about forecast quality and nothing else. The coin flip sits at 1.89 on these worlds. Nine of the ten forecasts carry the unshrunk fingerprint: the verifier found them closest to the tier that fits the right model but believes small samples, on most or all of the eight worlds. That is the failure the task was built around and the failure the ladder predicted, and it is the statistical mistake Section 5.4 measured in the real archive before any model saw the task. Run 8 needs a note. GPT-5.5 wrote a complete 230-line forecaster and then Codex reported that my account had reached its usage limit, and the session ended at 13 minutes. The program it had delivered was graded and failed at 1.362 with the same fingerprint as the others. I count the run and flag it, because the model did not get its full budget; excluding it leaves GPT-5.5 at 0 of 4 and changes no conclusion.

6.1 The near miss

GPT-5.5's third run, run 6, missed the bar by 0.9 points on all eight worlds and by 1.7 on the held-out seven. Its forecasts resemble the reference on seven worlds, so it regularised properly and lost on something smaller. A tolerance of 1.12 would have passed it. I report that sensitivity rather than act on it: the rule was fixed before the run, and a bar that moves after a score is seen is not a bar. The margin is of the same order as the reference's own advantage, which is why Section 5.13 states that limitation as plainly as it does.

6.2 What the programs did, and how they checked themselves

I read the six programs from the first round and the closing message of each session. All six fit the documented ball model by penalised likelihood with the exact gradient, build a skill book, and simulate on the shipped engine. The architecture is right in every one. What differs is the prior scale, and every one of the six set it too weak by a large factor without a check that could have seen it. The truth for batter quality is a spread of 0.15, a ridge of 44.

Trial	Model	Prior on batter quality as written	Against the truth	Other choices	Self-check before submitting
1	Opus 4.7	one ridge of 1.0 on every block	44 times too weak	day spread from unshrunk per-match effects; wear folded into dew	runtime, determinism, dependencies. None on accuracy.
2	GPT-5.5	prior sd 0.75, plus a per-season form sd 0.32	5 times too wide, 25 in variance	recency weight 0.82 per season; day spread fixed at 0.28	syntax, output shape, runtime. None on accuracy.
3	Opus 4.7	ridge 1.0 on style and quality, 2 on bowler quality	44 times too weak	synthetic leagues generated under these same priors	sharp, against the wrong world. See below.
4	GPT-5.5	ridge 1.2 on quality, 2.0 on style, 1.5 on bowler quality	37 times too weak	recency 0.72 per season; per-season form blocks	runtime, determinism, columns. None on accuracy.
5	Opus 4.7	ridge 4 on quality, style and bowler quality; 40 on day effects	11 times too weak on quality; day over-shrunk	closest priors of the five failures	symmetry (identical sides give 0.496) and forecast range. None on accuracy.
6	GPT-5.5	prior sd 0.45 per role plus 0.38 per player and bowling style	about 3 times too wide	recency 0.55 per season, and every forecast's logit multiplied by 0.90	compile and run only.

Trial 3 is the one the design turns on. It did what the handbook points to: it built synthetic leagues with the shipped engine, scored itself against their known truth, and reported that it sat at 0.34 times the coin flip, "0.64 to 0.67 times the estimated reference, well inside the 1.10 tolerance". It failed at 1.57. Its synthetic leagues were generated with player spreads of about 1.0, the inverse of its own ridge, against a truth of 0.15 to 0.35. In a world where players differ that much any estimator looks good. The check was sharp and aimed at a world that already agreed with its assumptions, so it measured how well the forecaster recovers its own priors. Verification is necessary and not sufficient; the check has to be tied to the data in hand. Trial 6, the near miss, is the other lesson. Its priors are still too wide, but it multiplies every finished forecast's logit by 0.90, pulling everything toward one half. That hedge compensates for part of the overconfidence the priors create, which is why it resembles the reference on seven worlds. It did not regularise correctly; it regularised badly and softened the output.

6.3 The ablations

A fingerprint is a lead, not a cause. To confirm the diagnosis by intervention I reran each program on one graded world, held-out world c, with exactly one constant changed, and scored it against that world's truth and stored reference. The first row of each pair reproduces the verifier's own number for that world to the last digit, which shows the reruns are of the same programs the grader saw. The reference's regret on this world is 0.0074 and the coin flip sits at 1.26 times it, so it is a quiet world where confident error is expensive.

Trial	As submitted	The one change	After
1, Opus	2.70	the single ridge of 1.0 on every block raised to 25	1.36
2, GPT-5.5	2.55	every prior spread multiplied by 0.33	1.57
3, Opus	2.17	every ridge multiplied by 12	1.64
4, GPT-5.5	2.29	every ridge multiplied by 12	1.60
5, Opus	2.10	the two quality ridges raised from 4 to 44 and 31, the true values	1.27
6, GPT-5.5	1.42	the output hedge removed (logit scale 0.90 to 1.0)	1.67

Trial	As submitted	The one change	After
6, GPT-5.5	1.42	the output hedge deepened (0.90 to 0.75)	1.13
6, GPT-5.5	1.42	every prior spread halved, hedge kept	0.91

Regret as a multiple of the reference's on held-out world c. Every change moved the number the way the diagnosis predicted, and by a lot: five programs cut their excess regret by half or more on the strength of one constant. None of the five reaches the reference on one change, because each also carries a second wrong choice, an unshrunk day spread, a recency weight, a form block, wear folded away, and those were left untouched on purpose. Trial 6 is the clean case. Removing its hedge made it worse, deepening the hedge made it better, and correcting its priors while keeping the hedge made it beat the reference on this world. The near miss was a softened wrong answer, and a correctly regularised version of that program would very likely pass. That is the strongest statement the evidence supports about the bar: it sits where the task intended, one honest check away from a pass.

7. Experiment log

Experiment	Question	Result	Decision
Pilots of Architecture A at 8, 24 and 48 scripts	Does a documented rule at scale defeat the top tier?	Opus 4.7 and GPT-5.5 scored 1.000 every time	Keep A as a fallback. Stop scaling it.
Smaller models on A	Does A discriminate at all?	Sonnet 5 and Haiku 4.5 scored 0 of 24 for different reasons	The verifier and its fingerprints work.
Mutation zoo for B	Does exhaustive schedule exploration catch each defect?	All five caught, each with its own fingerprint	The method is sound. B parked on expected difficulty.
Split halves on the archive	How much of a season is skill?	Batting reliability 0.46, bowling 0.27; last three innings weight -0.02	Shrinkage is the heart of the task. No hot streaks.
Interactions on the archive	Are head-to-head records signal?	Repeat 0.18 on 113 selected pairs, with levels removed within each half	Type-level effects only. Raw tables become a documented trap.
Player directions	In what ways do real players differ?	71 percent of batter variation on one axis: sixes up, ones and twos down, dismissals up	A hand-set direction replaced by the measured one.
First ladder, 109 matches of history	Can anyone beat the coin flip?	No forecaster did	History length is a dial. Use 270 matches.
State-response ablation	Does situational batting fix totals and chases?	Spread rose 25.4 to 30.3; chase rate rose, against the literature's claim	Keep the response. Add a wear constant.
Re-validation after raising wear	Does the raised constant hit the archive's chase rate?	Chasers win 0.510 against 0.509; mean 189.2; spread 35.0	Chase rate settled; spread of totals stays a limitation. A stale table corrected.
Ladder on three worlds	Is the bar placeable?	Reference 0.57 to 0.63; closest careless tier at 1.14 times	I also concluded the reference was stable, which was wrong.
Bar analysis on eight fresh worlds	What must the tolerance absorb?	Noise up to 5.9 percent per world; coin flip beats the reference on 1 of 8; totals clean	Bar moved to the total over eight worlds at 1.10. Committed before any pilot.

Experiment	Question	Result	Decision
Calibration rebuilt from raw data	Can the constants be reproduced?	Every block within 0.0007 of an earlier build; seed 101 reproduces that build's world byte for byte with its constants	The pipeline is reproducible. The generated world is sensitive to the fourth decimal.
Gates on the packaged task	Does the public path meet its own bar?	Oracle 1.000 on every key at exactly 1.000 times the reference; starter 0.000 overall	The task is solvable from the agent's files.
Pilots, five trials per model	Does a named model fail under the committed rule?	Opus 4.7 0 of 5 (1.53, 1.57, 1.40, 1.36, 1.70); GPT-5.5 0 of 5 (1.58, 1.55, 1.11, 1.36, 1.41)	Both named models fail, for the cause the ladder predicted.
One-constant ablations of the six first-round programs on held-out world c	Is the fingerprint the cause?	Every program moved as predicted on one constant: 2.70 to 1.36, 2.55 to 1.57, 2.17 to 1.64, 2.29 to 1.60, 2.10 to 1.27; the near miss beat the reference at 0.91 once its priors were halved	The cause is confirmed by intervention. The near miss was a hedged wrong answer.

8. Risks and limitations

- Five trials per model is a small sample. 0 of 5 leaves a true pass rate as high as about 45 percent inside a 95 percent interval, and one run missed by less than one point.
- The verdicts move with the bar. At 1.12 the near miss would have passed. The rule is relative, tied to named tiers and committed first, but it remains a choice.
- The reference's prior spreads were set by someone who knew the truth. The clean fix, a reference that learns its spreads from the data, is the next step, and it would need its own bar analysis and its own committed rule before any pilot.
- Every failure on this build reduces to one mistake, an unchecked prior scale, and the ablations confirm it. A skeptic can say the task measures whether an agent tunes a ridge penalty. My answer is that the capability underneath is building a check that can see your own error, which trial 3 shows is harder than it sounds, but it is one failure mode and the task claims no more.
- The handbook's yardstick sentence helps self-verification. I disclosed it for fairness, and it makes the task easier for an agent that uses it.
- The program-level reading and the ablations cover the six runs of the first round; the four later runs are attributed by fingerprint only, and all four carry the unshrunk fingerprint on every world. The ablations are on one graded world, and one constant each; they confirm direction and size, not the full accounting of every choice. One GPT-5.5 session was cut short by my account's usage limit after it had delivered its program.
- The league is synthetic. Its truth is the truth of my simulator, not of cricket. The spread of totals is 35.0 against the archive's 37.4; batter quality is weakly identified; the directions come from about a hundred regular players, so the quality axis is probably understated.
- Three of the six validation checks were tuning targets, so agreement on them is by construction.
- The engine has no fielding, no partnerships and no ball-to-ball memory beyond the scoreboard. The toss winner always chases and bowlers rotate in a fixed pattern. None of this affects grading, since the truth is computed under the same rules, but it bounds what the world claims about real cricket.

- Docker Desktop on macOS rejects the no-network mode for the verifier, so it is left undeclared. The verifier makes no network calls. The pilots ran on one platform; the oracle was also checked on linux/amd64 and matched the stored reference to the last printed digit.
- The brief names two model pairs: its goal line names GPT-5.5 and Claude Opus 4.7 and its opening section names a newer pair. These runs are of the pair the goal line and the command examples name. The newer pair would need its own runs on the frozen task, under the same committed rule.
- Harbor's log redaction replaces the literal true in each job's details.json with a placeholder, so those files need a one-word substitution before they parse; the reward files are untouched. The summaries in the run report were produced that way and say so.

9. What comes next

Step	What it produces	Status
Ablations of runs 7 to 10, and on a second graded world	The same confirmation for the four later runs, and a check that the size of the effect holds on a world with more to predict	Open; the method and script are in place
More trials per model	A narrower interval on the pass rate than 0 of 5 gives	Five per model done
A reference that learns each skill's spread from the world	Removes the reference's advantage; needs a fresh bar analysis and a fresh committed rule before any pilot	Measured earlier at six to seven percent lower regret. Not adopted under the current rule.
An off-season circuit with its own hidden scoring level	A second source of evidence per player, and a league-equivalence problem an agent must solve	Designed. On real data, a player's internationals in the year before an IPL season add five to six points of explained variance.
A fresh-clone rebuild on another machine	Independent confirmation of the package	Open
The newer model pair the brief's opening section names	Whether the same failure holds one generation on	Open; needs access to both harnesses

Appendix. Repository map, bugs the checks caught, sources

Path	What it holds
dev/parse_archive.py, explore_overs.py, explore_reliability.py, fit_state.py, fit_constants.py, build_calibration.py	Archive to league/calibration.json. Aggregates only.
league/calibration.py, engine.py, world.py, league_io.py	Measured and chosen constants; the shared engine; the generator and the truth engine; the public file reader
forecasters/ladder.py, scoring/exact.py	The reference ladder and the exact scorer
dev/validate_world.py, run_ladder.py, bar_analysis.py, make_task_data.py	Validation, the ladder, the bar analysis, the eight graded worlds
harbor/, task_src/, package_task.py, Makefile	Task metadata, containers, verifier, oracle, handbook, starter, and the packager

Path	What it holds	
task_data/, dist/, jobs/	The shipped worlds and private truth; the packaged task; the gate and pilot records	
NOTES.md, DECISIONS.md, RUN_REPORT.md, docs/ARCHITECTURE.md	What each file does and why, with checked citations; the rule and its history; the runs; the diagram	

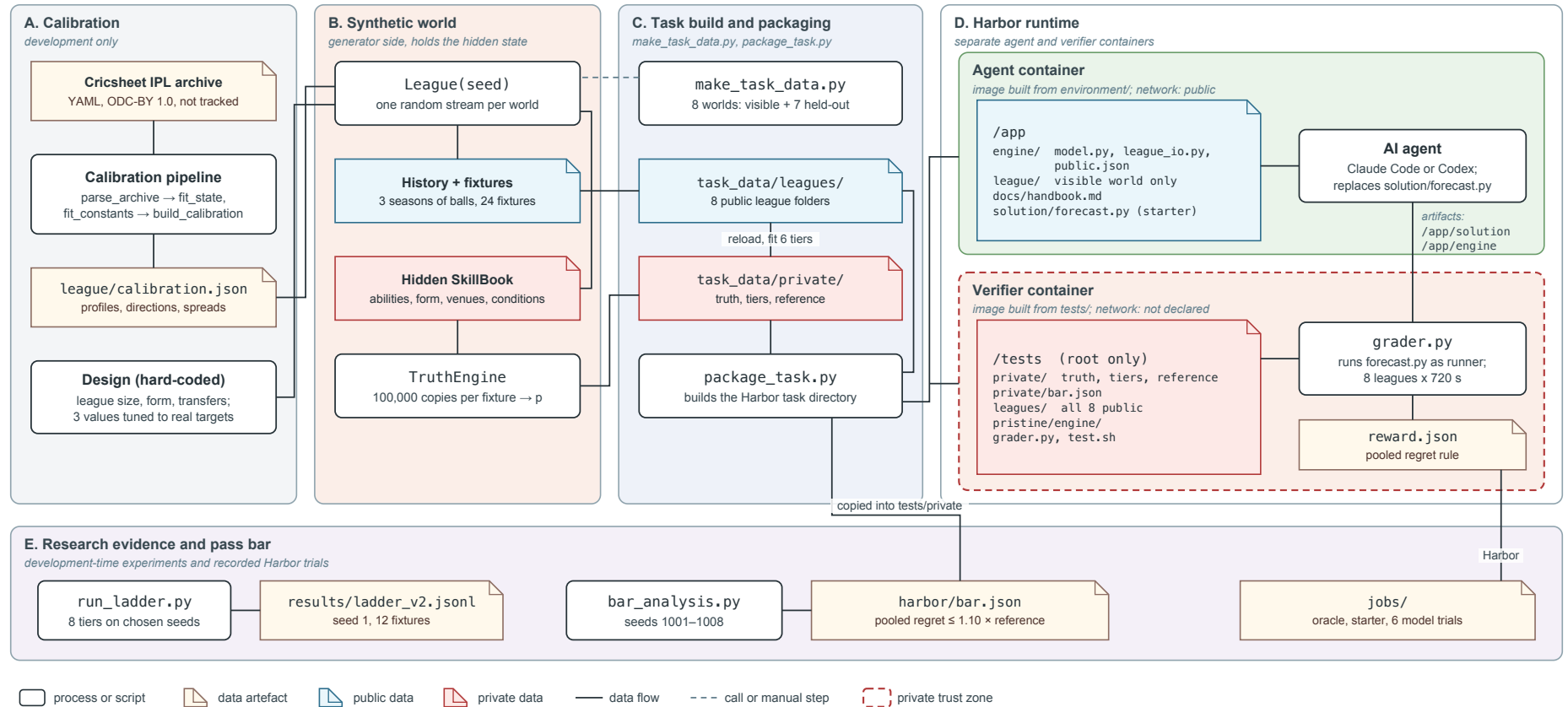
File	What was typed	How it was caught
parse_archive.py	run_out for run out	12 bowler wickets expected, 14 printed
explore_overs.py	the subtraction that makes a count before the ball	125,465 legal balls expected, 717 short
fit_state.py	plus 0.01 plus theta for plus 0.01 times theta in the gradient	converged in 128 iterations instead of 145, two rows off by 0.04
engine.py	a missing class; RUN for RUNS	the first import; the first innings
league_io.py	a missing return	the first call returned None
ladder.py	a method header indented one level too deep	an indentation error on the line after it
solve.sh	spaces around a shell assignment	the oracle gate scored the starter; bash -n had passed the file
scoring/exact.py, my check	a guessed expected value	the code was right and the guess was wrong
this document	a validation table from before the wear constant changed	rerunning the validation script

Sources. Calibration constants derived from data sourced from Cricsheet (cricsheet.org), licensed under the Open Data Commons Attribution License (ODC-BY 1.0). Good (1952) and Gneiting and Raftery (2007) for proper scoring rules; Kullback and Leibler (1951) for the divergence; Selten (1998) for the case against the log score; Murphy (1973, 1988) for skill scores; Cox (1958) for the spread slope; Morris, White and Crowther (2019) for simulation studies with known truth; Wickham (2014) for tidy data; the MCC Laws of Cricket, 2017 Code, for overs, wides, no-balls and consecutive overs; Spearman (1904, 1910) and Brown (1910) for reliability and disattenuation; Lord and Novick (1968) for classical test theory; Galton (1886), James and Stein (1961), Efron and Morris (1977) and Hoerl and Kennard (1970) for shrinkage; Stone (1974) for cross-validatory choice; McFadden (1974) for the multinomial logit; Liu and Nocedal (1989) and Byrd, Lu, Nocedal and Zhu (1995) for L-BFGS; Blanchard, Higham and Higham (2021) for the shifted softmax; Anscombe (1956) for the continuity correction; Tipping and Bishop (1999) and Fuller (1987) for noise-corrected components; Tango, Lichtman and Dolphin (2007) for batter-pitcher matchups; Uhlenbeck and Ornstein (1930) and Glickman (1999) for the form process; Davis, Perera and Swartz (2015), Swartz, Gill and Muthukumarana (2009) and Duckworth and Lewis (1998) for cricket simulation and resources; Metropolis and Ulam (1949) for Monte Carlo; Devroye (1986) for inversion sampling; Bradley and Terry (1952) for paired comparisons; Makridakis, Spiliotis and Assimakopoulos (2020) for baselines; Sargent (2013) and Law (2015) for simulation validation; Dwork et al. (2015) for holdouts; Simmons, Nelson and Simonsohn (2011) and Nosek et al. (2018) for fixing the rule first; Saltzer and Schroeder (1975) for least privilege; Amodei et al. (2016) and Skalse et al. (2022) for reward hacking; Peng (2011) for reproducible research; Shafranovich (2005) for CSV. Every citation was checked against library records this week; the full list with volumes and pages is in NOTES.md.

Plates

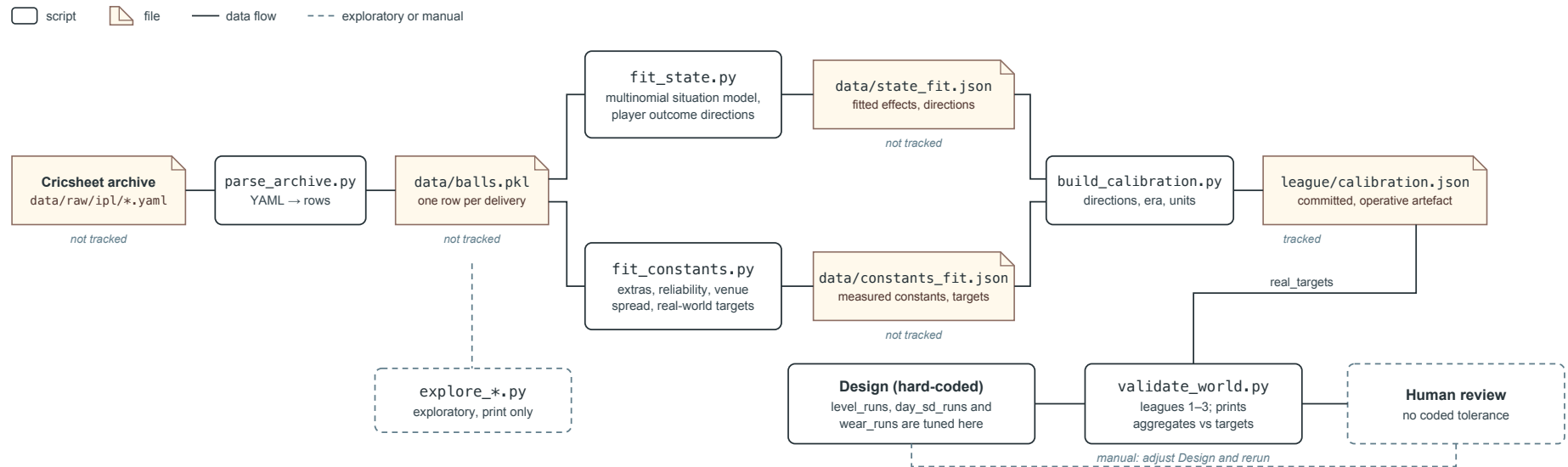
Seven drawings of the system as it is, generated from the repository by the scripts under `figures/`. Each is a vector drawing; zoom in for the labels.

Plate 1. The whole system



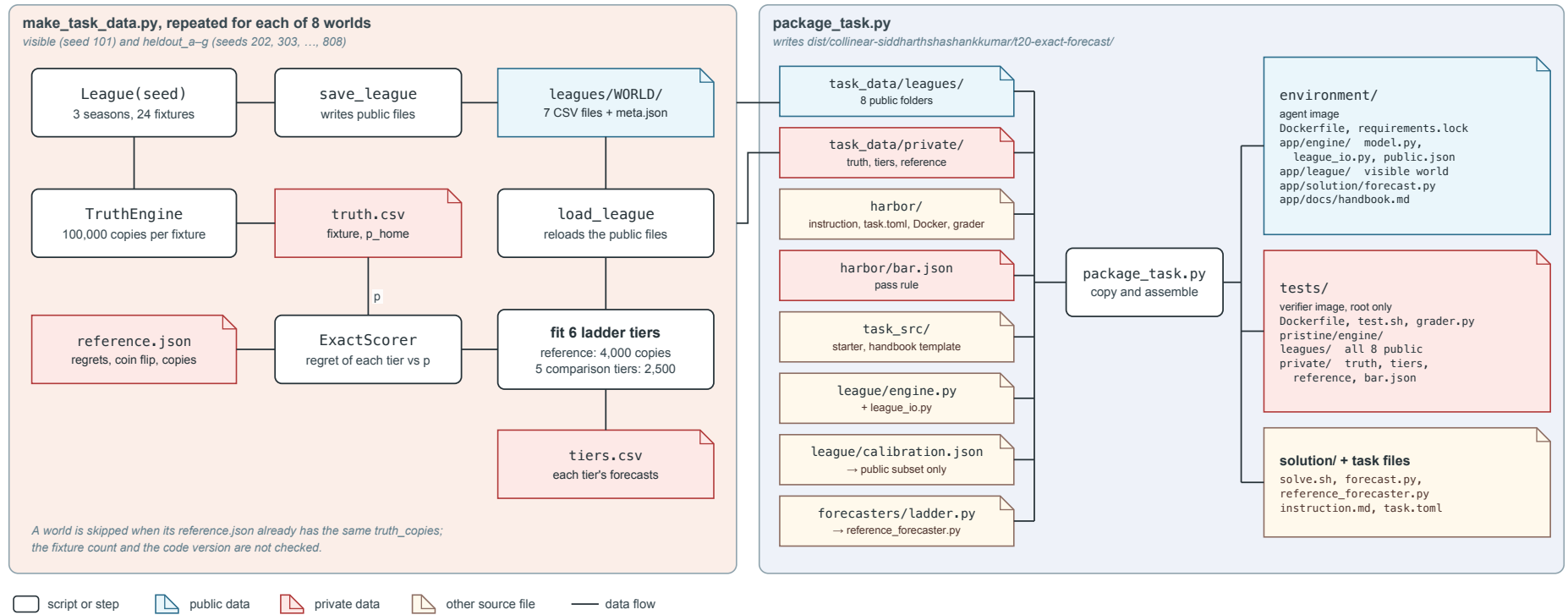
Five zones: calibration from the real archive, the synthetic world that holds the hidden state, the task build and packaging, the Harbor runtime with its separate agent and verifier containers, and the research evidence behind the pass bar.

Plate 2. From the archive to the constants file



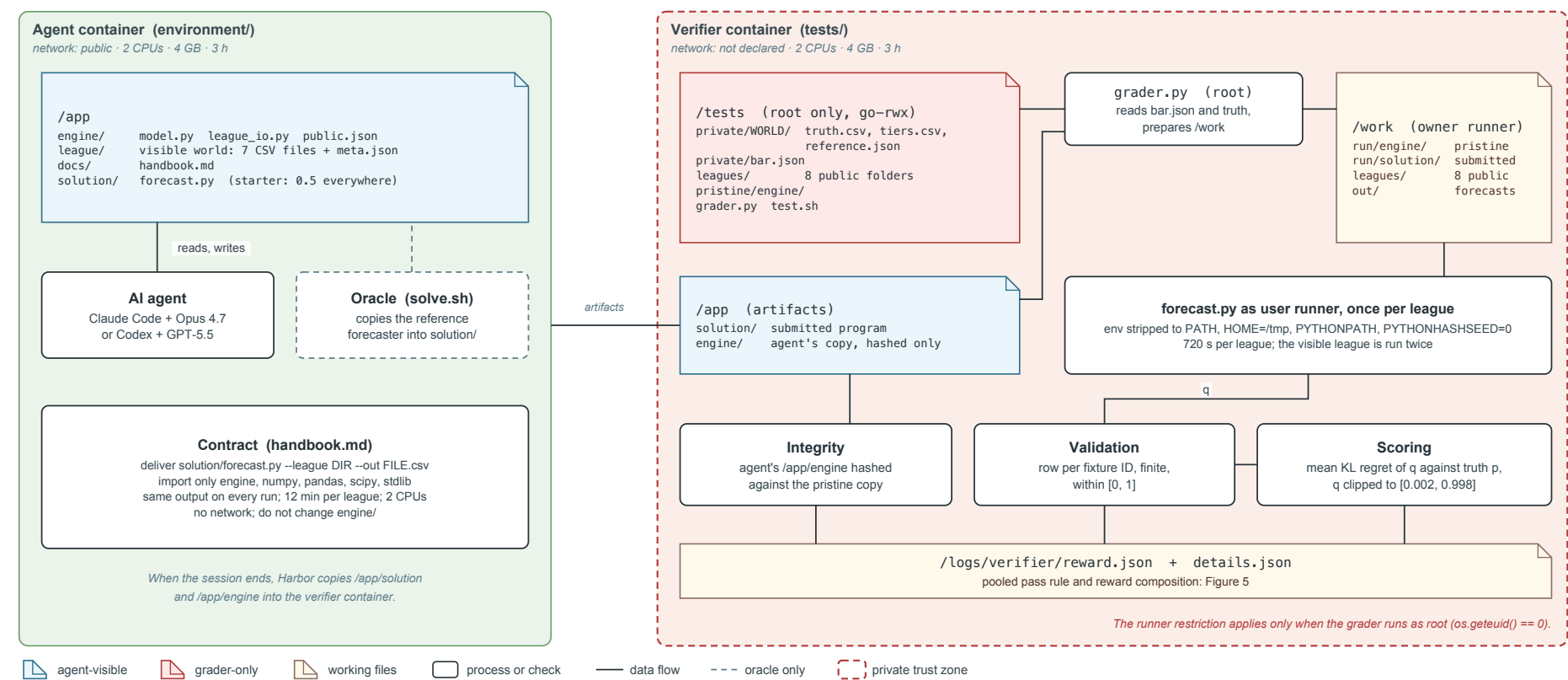
The six development scripts, what each writes, which files are tracked, and the manual loop in which three design constants are adjusted against the validation targets.

Plate 3. Building the eight worlds and packaging the task



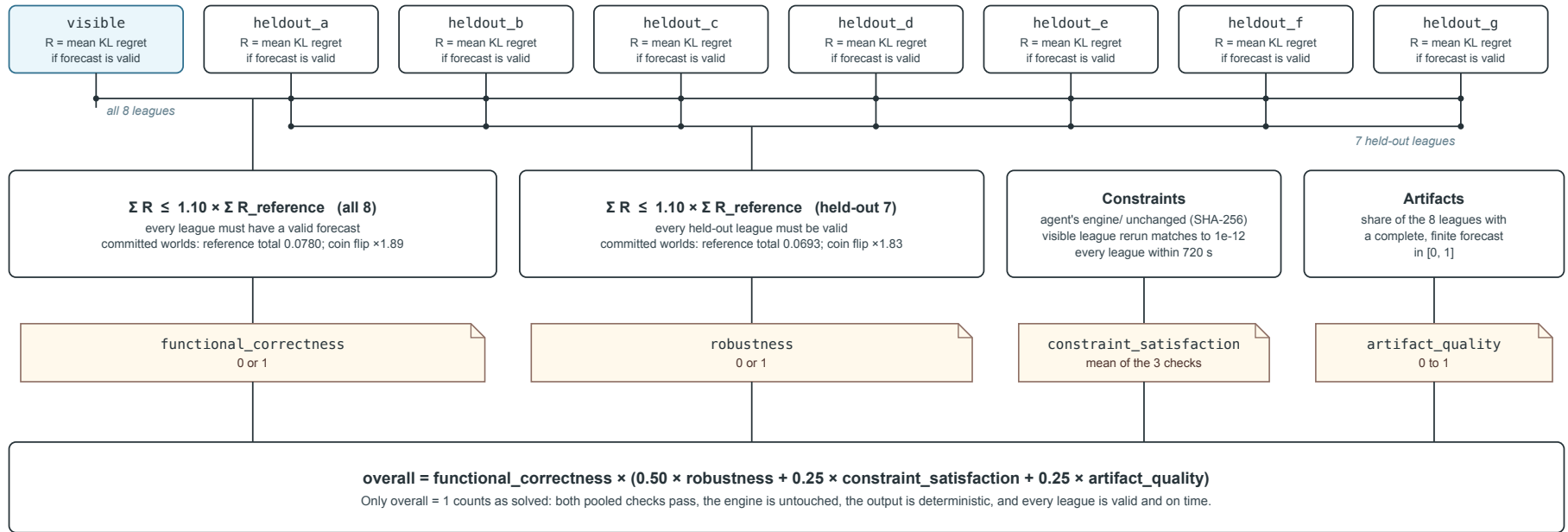
Left, what `make_task_data.py` does for each world, including the reload of the public files before the tiers are fitted. Right, the four source trees `package_task.py` assembles into the agent side, the verifier side and the oracle.

Plate 4. The two containers and the grader's checks



The agent sees only the public task. The verifier holds the answers, runs the submission as an unprivileged user beside a pristine engine, and applies the integrity, validity and scoring checks.

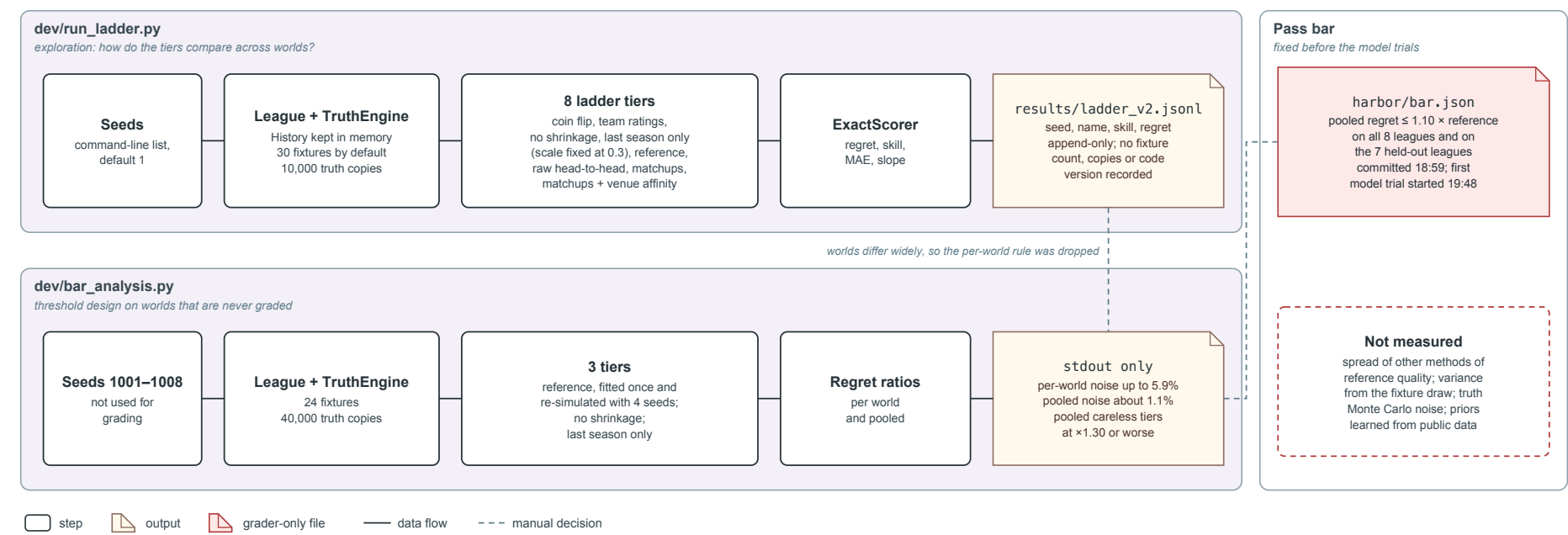
Plate 5. The pass rule and the reward composition



$R_{\text{reference}}$ comes from reference.json (the reference forecaster at 4,000 copies per fixture). Forecasts are clipped to $[0.002, 0.998]$ before scoring.

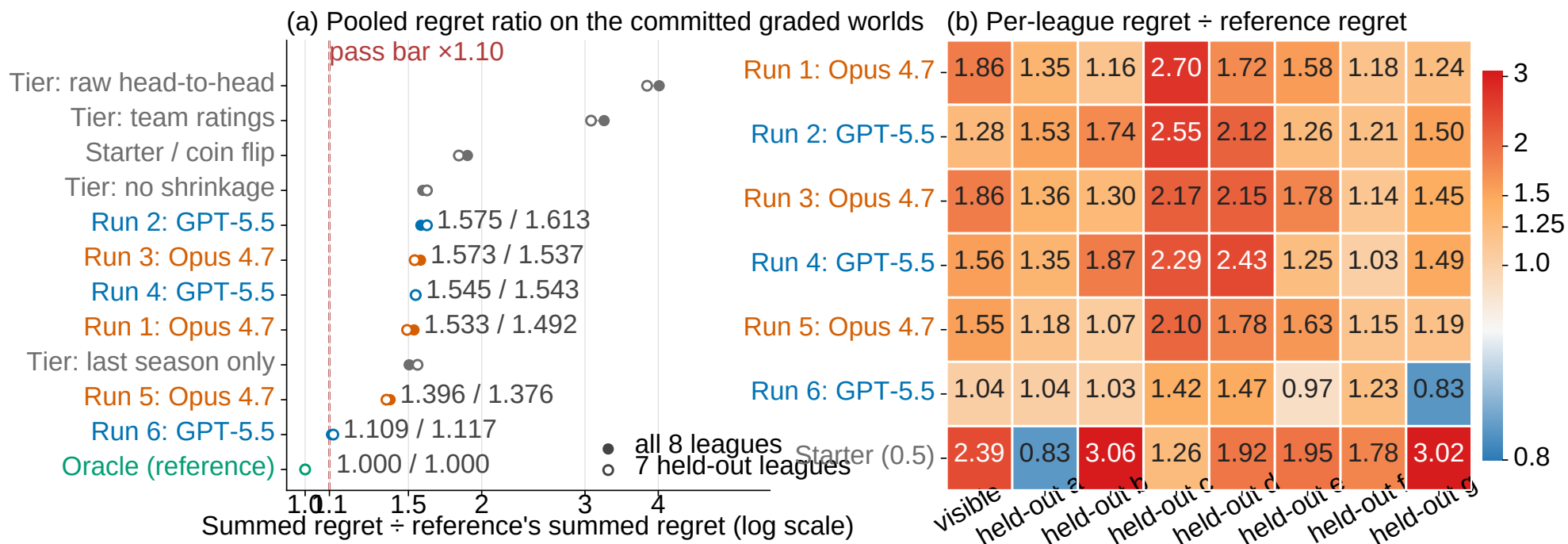
One rule applied twice, to all eight worlds and to the seven held-out worlds, then the constraint and artifact checks, and how the four keys combine into the overall reward.

Plate 6. The experiments behind the bar



The ladder runner across worlds, the bar analysis on eight worlds that are never graded, the quantities each measured, and the rule they produced.

Plate 7. Pilot results on the graded worlds



Left, the six first-round model runs and the ladder tiers as multiples of the reference's summed regret, against the 1.10 bar. Right, the same runs world by world. Drawn from the job records of the first round; runs 7 to 10 are in the table in Section 6.